

SKRENO AUTOMATION · PLAYWRIGHT + TYPESCRIPT

Skreno অটোমেশন শুরু করার ধাপে ধাপে গাইড

ফাঁকা Playwright প্রজেক্ট থেকে Page Object Model, login session, test data আর CI পর্যন্ত। প্রতিটা ধাপে কী করবেন আর কেন করবেন, মেন্টরের ভাষায় বাংলায় বোঝানো।

প্রজেক্ট: `~/skreno-automation` · Playwright 1.64 · Node 22

সূচিপত্র

1. সেটআপ ঠিকমতো কাজ করছে কিনা যাচাই
2. Git চালু করা
3. ফোল্ডার স্ট্রাকচার (Page Object Model)
4. `.env` দিয়ে গোপন তথ্য আলাদা রাখা
5. `playwright.config.ts` আপডেট
6. `package.json` এ script যোগ
7. Codegen দিয়ে locator খোঁজা
8. প্রথম Page Object: LoginPage
9. প্রথম test লেখা
10. একবার login, বারবার ব্যবহার (storageState)
11. Faker দিয়ে test data
12. Prettier কনফিগার
13. CI (GitHub Actions) চালু
14. মেন্টরের কিছু পরামর্শ

বর্তমান অবস্থা: প্রজেক্টটা একদম নতুন Playwright + TypeScript সেটআপ। ভেতরে আছে @playwright/test, faker, dotenv, eslint, prettier, একটা example test আর একটা GitHub Actions workflow। ব্রাউজারগুলো আগে থেকেই ইনস্টল করা আর Node v22 চলছে, তাই সব ঠিক আছে। এখনো git init করা হয়নি। নিচের ধাপগুলো ক্রমানুসারে নিজের হাতে করুন।

1 সেটআপ ঠিকমতো কাজ করছে কিনা যাচাই করুন

```
cd ~/skreno-automation
npx playwright test --project=chromium
npx playwright show-report
```

কেন: নিজের কোড লেখার আগে নিশ্চিত হয়ে নিন যে টুলটা ঠিকমতো চলছে। example test পাস করলে বুঝবেন Node, Playwright আর ব্রাউজার সব ঠিক আছে। পরে কোনো সমস্যা হলে তখন জানবেন যে দোষটা আপনার কোডে, সেটআপে না।

2 Git চালু করুন

```
git init
git add .
git commit -m "Initial Playwright setup"
```

কেন: অটোমেশন প্রজেক্ট দিন দিন বড় হবে। কিছু ভেঙে গেলে আগের অবস্থায় ফিরে যাওয়ার জন্য git লাগবে। .gitignore আগেই আছে, তাই node_modules আর report ফাইলগুলো commit হবে না।

3 ফোল্ডার স্ট্রাকচার তৈরি করুন (Page Object Model)

```
mkdir -p pages tests/auth tests/dashboard fixtures utils test-data
rm tests/example.spec.ts
```

ফোল্ডারগুলো এরকম দাঁড়াবে:

pages/	→ প্রতিটা পেজের locator আর action (LoginPage.ts, DashboardPage.ts)
tests/	→ আসল test ফাইল (*.spec.ts)
fixtures/	→ custom fixture, যাতে প্রতিটা test এ বারবার new LoginPage() লিখতে না হয়
utils/	→ helper function (যেমন faker দিয়ে data বানানো)
test-data/	→ স্থির JSON data

কেন: এখানে মেন্টর হিসেবে সবচেয়ে জরুরি পরামর্শটা দিই। test ফাইলের ভেতরে সরাসরি locator লিখবেন না। Skreno-র UI বদলালে ৫০টা test ফাইল ঠিক করতে হবে, যেখানে POM থাকলে শুধু একটা page ফাইল ঠিক করলেই হয়। এটাকে বলে **Page Object Model**, আর ইন্ডাস্ট্রিতে এটাই স্ট্যান্ডার্ড।

4

`.env` ফাইল দিয়ে গোপন তথ্য আলাদা রাখুন

প্রজেক্টের root এ `.env` নামে একটা ফাইল বানান:

```
BASE_URL=https://skreno.ai
TEST_EMAIL=your-test-account@example.com
TEST_PASSWORD=your-password
```

তারপর `.gitignore` এ নিচের লাইনটা যোগ করুন:

```
.env
```

চাইলে একটা `.env.example` ফাইলও রাখতে পারেন। এতে value থাকবে না, শুধু key গুলো থাকবে, যাতে টিমের অন্যরা বুঝতে পারে কোন কোন variable লাগবে।

কেন: পাসওয়ার্ড কখনো কোডে বা git এ রাখবেন না। এছাড়া `.env` থাকলে staging আর production এর মধ্যে শুধু URL বদলে একই test চালাতে পারবেন।

5

`playwright.config.ts` আপডেট করুন

ফাইলে তিনটা জিনিস বদলাতে হবে।

ক) উপরের কमेंট করা `dotenv` লাইনগুলো চালু করুন

```
import dotenv from 'dotenv';
import path from 'path';
dotenv.config({ path: path.resolve(__dirname, '.env') });
```

খ) `use` ব্লকটা এভাবে লিখুন

```
use: {
  baseUrl: process.env.BASE_URL,
  trace: 'retain-on-failure',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
},
```

গ) `projects` এ আপাতত শুধু `chromium` রাখুন

firefox আর webkit ব্লক দুটো কमेंট করে দিন।

কেন:

- baseUrl সেট করলে test এ পুরো URL না লিখে শুধু `page.goto('/login')` লিখলেই হবে।
- test fail করলে screenshot, video আর trace পাবেন। ডিবাগ করার সময় এগুলো সবচেয়ে বেশি কাজে আসে।
- শুরুতে তিন ব্রাউজারে চালালে সময় তিনগুণ লাগবে। আগে chromium এ stable করুন, পরে বাকিগুলো চালু করবেন।

6 package.json এ script যোগ করুন

"scripts": {} অংশটা এভাবে বদলে দিন:

```
"scripts": {  
  "test": "playwright test",  
  "test:headed": "playwright test --headed",  
  "test:ui": "playwright test --ui",  
  "test:debug": "playwright test --debug",  
  "report": "playwright show-report",  
  "codegen": "playwright codegen"  
}
```

কেন: লম্বা কমান্ড বারবার টাইপ করার দরকার পড়বে না, শুধু `npm run test:ui` লিখলেই হবে। **UI mode** নতুনদের জন্য সবচেয়ে কাজের। এখানে প্রতিটা স্টেপ চোখে দেখতে পারবেন, আবার time-travel করে আগের স্টেপেও ফিরে যেতে পারবেন।

7 Codegen দিয়ে locator খুঁজে বের করুন

```
npx playwright codegen https://skreno.ai
```

একটা ব্রাউজার খুলবে। সেখানে আপনি login করুন, ক্লিক করুন। Playwright নিজে থেকেই সেই কাজগুলোর কোড লিখে দেবে।

কেন: Skreno-র কোন বাটনের জন্য কোন locator লাগবে, সেটা এখান থেকে শিখবেন।

সতর্ক থাকুন: codegen এর কোড সরাসরি কপি করে test বানাবেন না। শুধু locator গুলো নিয়ে page class এ বসাবেন। `getByRole` আর `getByLabel` ধরনের locator ব্যবহার করুন। CSS class দিয়ে বানানো locator বাদ দিন, কারণ ওগুলো সহজে ভেঙে যায়।

```
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Email'); // codegen থেকে আসল locator বসান
    this.passwordInput = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Log in' });
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

কেন: এখন থেকে test গুলোতে শুধু `loginPage.login(...)` লিখবেন। এতে test পড়তে সাধারণ ইংরেজির মতো লাগে, আর locator শুধু এক জায়গাতেই থাকে।

9 প্রথম test লিখুন: tests/auth/login.spec.ts

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../../pages/LoginPage';

test.describe('Login', () => {
  test('valid user can log in', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login(process.env.TEST_EMAIL!, process.env.TEST_PASSWORD!);
    await expect(page).toHaveURL(/dashboard/);
  });

  test('wrong password shows error', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login(process.env.TEST_EMAIL!, 'wrong-pass');
    await expect(page.getByText(/invalid|incorrect/i)).toBeVisible();
  });
});
```

চালান:

```
npm run test:ui
```

কেন: প্রতিটা ফিচারের জন্য একটা positive আর অন্তত একটা negative test লিখবেন। এটা ভালো QA-র অভ্যাস। আর খেয়াল করুন, expect ছাড়া কোনো test আসলে কিছুই যাচাই করে না।

10 Login একবার করে সেটা বারবার ব্যবহার করুন (storageState)

এটা একটু advanced ধাপ, ধাপ ৯ পাস করার পরে করবেন।

tests/auth.setup.ts ফাইল

```
import { test as setup } from '@playwright/test';
import { LoginPage } from '../../pages/LoginPage';

setup('authenticate', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login(process.env.TEST_EMAIL!, process.env.TEST_PASSWORD!);
  await page.waitForURL(/dashboard/);
  await page.context().storageState({ path: 'playwright/.auth/user.json' });
});
```

তারপর config এর projects অংশ এভাবে লিখুন

```
projects: [
  { name: 'setup', testMatch: /\.*\setup\.ts/ },
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/user.json' },
    dependencies: ['setup'],
  },
],
```

কেন: ধরুন dashboard এর ১০০টা test আছে। প্রতিটিতে আলাদা করে login করলে সময় নষ্ট হবে। এই পদ্ধতিতে login হয় একবার, তারপর সেই cookie সব test ব্যবহার করে। .gitignore এ /playwright/.auth/ আগে থেকেই আছে, তাই session ফাইল git এ যাবে না।

11 Faker দিয়ে test data বানান: `utils/dataFactory.ts`

```
import { faker } from '@faker-js/faker';

export const newUser = () => ({
  name: faker.person.fullName(),
  email: faker.internet.email(),
  password: faker.internet.password({ length: 12 }),
});
```

কেন: signup বা ফর্মের test এ hardcoded data দিলে দ্বিতীয়বার চালানোর সময় "email already exists" এরর আসবে। Faker প্রতিবার নতুন data বানায়, তাই test গুলো একটা আরেকটার ওপর নির্ভর করে না।

12 Prettier কনফিগার করুন

root এ .prettierrc ফাইল বানান:

```
{ "singleQuote": true, "semi": true, "printWidth": 100 }
```

package.json এর scripts এ যোগ করুন: "format": "prettier --write ."

কেন: কোডের স্টাইল সবসময় একরকম থাকবে। ESLint 10 এর জন্য typescript-eslint আলাদা ইনস্টল করতে হয়, ওটা পরে করলেও চলবে। আপাতত Prettier যথেষ্ট।

13 CI (GitHub Actions) চালু করুন

`.github/workflows/playwright.yml` আগে থেকেই আছে। GitHub এ repo push করার পরে:

1. **Settings** → **Secrets and variables** → **Actions** এ যান।
2. `BASE_URL`, `TEST_EMAIL`, `TEST_PASSWORD` নামে তিনটা secret যোগ করুন।
3. workflow এর test চালানোর স্টেপে এটা যোগ করুন:

```
env:  
  BASE_URL: ${ secrets.BASE_URL }  
  TEST_EMAIL: ${ secrets.TEST_EMAIL }  
  TEST_PASSWORD: ${ secrets.TEST_PASSWORD }
```

কেন: CI তে `.env` ফাইল থাকে না, তাই secrets থেকে value পাঠাতে হয়। এরপর প্রতিটা push এ test নিজে থেকেই চলবে।

★ মেন্টরের কিছু পরামর্শ

1. **ছোট থেকে শুরু করুন।** প্রথম সপ্তাহে শুধু login, dashboard লোড হওয়া আর logout, এই তিনটা ফ্লো stable করুন।
2. `page.waitForTimeout()` **কখনো ব্যবহার করবেন না।** Playwright নিজে থেকেই অপেক্ষা করে (auto-wait)। কিছু দেখা যাওয়া পর্যন্ত অপেক্ষা করতে চাইলে `expect(...).toBeVisible()` লিখুন।
3. **Test fail করলে আগে trace দেখুন।** `npx playwright show-trace` চালালে কোন স্টেপে কী হয়েছিল সব দেখা যায়।
4. **প্রতিটা test স্বাধীন রাখুন।** একটা test আরেকটার ফলাফলের ওপর নির্ভর করবে না।
5. **Chrome extension এর test পরে।** Skreno extension test করতে `launchPersistentContext` লাগে। ওটা আলাদা ধাপ, আগে web অংশ শক্ত করুন।

কোনো ধাপে আটকে গেলে এররটা নিয়ে আসুন, তখন একসাথে ঠিক করব। 🚀